



Github 사용법

[Git Branch 전략](#)

[Git Commit 컨벤션](#)

[Github Issue 컨벤션](#)

[제목 작성](#)

[본문 작성](#)

[개발 브랜치 생성 가이드](#)

[Pull Request 컨벤션](#)

[제목 작성 가이드](#)

[본문 작성 가이드](#)

[리뷰 및 병합 규칙](#)

[개발 브랜치 → dev브랜치 병합 가이드](#)

[Git 사용법](#)

[git Branch 이동](#)

Git Branch 전략

- main
 - 개발 최종버전.
모든 개발 및 테스트 완료 후 dev브랜치로부터 병합
- dev
 - 개발 중 사용할 브랜치
개발중인 모든 기능 병합, 각 기능개발 브랜치 별로 dev브랜치에 PR을 통해 병합
- 개발 브랜치
 - 이슈에 따라 별도의 개발 브랜치를 생성하여 개별적으로 작업
 - 네이밍 규칙
 - 작업 유형 태그(feats, fix 등)와 이슈 번호 포함
 - 예시
`feat/#12` : 이슈 #12의 개발 작업
`fix/#14` : 이슈 #14의 버그 또는 기능 수정 작업

Git Commit 컨벤션

- 커밋 메세지 작성 시 아래 형식 사용

태그	설명	예시
feat	새로운 기능 추가	<code>feat: 로그인 기능 추가</code>
fix	버그 수정	<code>fix: 로그인 예외 처리 버그 수정</code>
docs	README 등의 문서 수정	<code>docs: API 명세 업데이트</code>
style	코드 스타일 변경	<code>style: 코드 포매팅 개선</code>
refactor	기능 변경 없이 코드 내부 구조 리팩토링	<code>refactor: 로그인 처리 로직 리팩토링</code>
test	테스트 케이스 작성 혹은 수정	<code>test: 사용자 인증 로직 테스트 케이스 추가</code>

태그	설명	예시
chore	라이브러리 버전 수정, 패키지 관리 등	chore: 의존성 버전 업데이트
comment	주석 추가 / 수정	comment: 불필요한 주석 제거
hotfix	배포된 버전에서의 급한 버그 수정	hotfix: 서버 Timezone 설정 변경
rename	파일, 클래스 등의 이름 변경	rename: UserController → AuthController 변경
remove	파일, 클래스 등의 삭제	remove: 사용하지 않는 DTO 제거
cicd	CI/CD 관련 설정	cicd: Github Actions workflow 추가

Github Issue 컨벤션

제목 작성

- Issue 제목은 작업의 유형과 내용을 한 눈에 이해할 수 있도록 작성
 - ex) [feat] 로그인 페이지 추가

본문 작성

[태그] 제목
(예: [feat] 로그인 기능 추가)

목적

- 해당 Issue를 생성한 이유
- ex) 회원 기능을 위한 로그인 페이지 추가

상세 내용

- 구현할 기능 또는 수정사항에 대한 구체적인 내용
- ex) 아이디, 비밀번호 입력 폼 추가
- ex) 로그인 기능 수정

추가 사항 (필수X)

- 관련 Issue 번호, 참고해야 할 사항 등

[fix] 페이지 레이아웃 수정 + Route 추가 #59

Issue details for [fix] 페이지 레이아웃 수정 + Route 추가 #59:

- Assignees:** limtjdghks
- Labels:** DESIGN, FEAT, REFACTOR
- Type:** Feature

목적

- 구현된 페이지 폰트 및 레이아웃 수정
- 페이지 Route 추가

상세 내용

- Recruit 페이지들 폰트 수정
- Recruit 페이지들 코드 구조 변경
- FAQ 페이지 레이아웃 및 텍스트 수정
- Main 페이지 지원하기 onClick 추가

- Issue 생성시 자기자신을 Assignee
- Issue 타입에 맞는 Label 설정
 - 라벨과 같은 Type 설정
(기능 개발 도중 fix와 Refactor가 있다면 Type은 Feature로, 수정사항은 커밋메세지에 fix로 작성)

개발 브랜치 생성 가이드

- 현재 브랜치를 dev 브랜치로 이동
 - `git checkout dev`
- git 명령어를 통해 현재 브랜치를 복제하여 새로운 브랜치 생성
 - `git checkout -b feat/#12`

Pull Request 컨벤션

제목 작성 가이드

- Issue와 같은 제목 사용

본문 작성 가이드

```
# [태그] 제목
(예: [feat] 로그인 기능 추가)
---

## Issue
- 해당 PR과 관련된 이슈 번호
- ex) #12

## 변경 내용
- 이번 PR에서 어떤 변경이 이루어졌는지 간략하게 작성
- ex) 로그인 API 연동, 로그인 페이지 onClick 이벤트 추가

## 구현 사항
- 새로운 기능 구현 내용 및 수정 경위 상세하게 작성
- ex) 로그인 예외처리 추가,

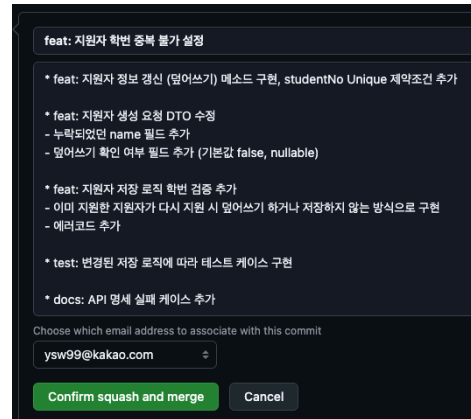
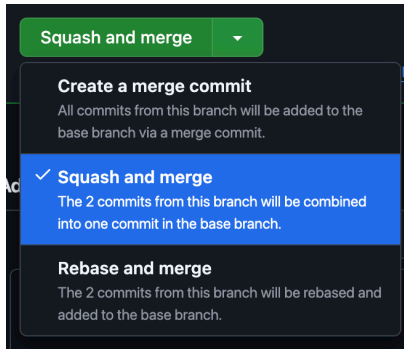
## 참고 사항
- 추가적으로 검토가 필요한 사항이나 팀원들이 알고 있어야할 내용 작성
- ex) Recruit 페이지 textarea 스크롤 해제하여 보기 편하게 변경
```

리뷰 및 병합 규칙

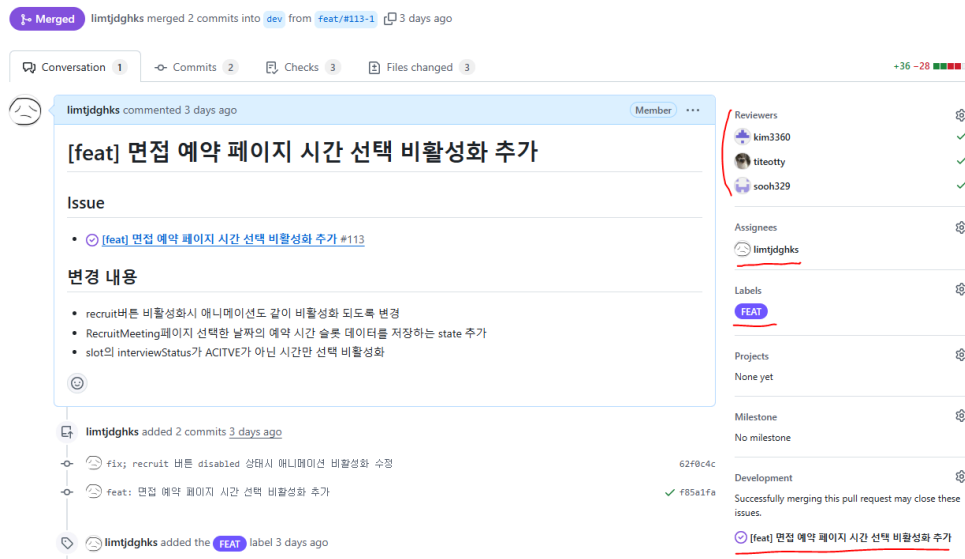
- 모든 팀원을 리뷰어로 assign 한다.
- 변경 사항에 대해 팀원 절반 이상의 승인을 받은 후 최종 병합한다.
- 코드에 수정이 필요한 이슈가 발견된 경우, 해당 사항을 개선한 후(commit) 승인을 얻은 후 병합

개발 브랜치 → dev브랜치 병합 가이드

- merge 타입은 Squash and Merge 사용



[feat] 면접 예약 페이지 시간 선택 비활성화 추가 #114



- PR 작성시 Reviewers에 팀원들 설정
- Assignees은 자기자신 설정
- Label은 Issue Label과 같이 설정
- PR 등록 후 Development에 관련된 Issue 설정

! merge 초록색 말고 빨간색 창으로 바뀌면서 에러 발생시 꼭 카톡으로 얘기할 것

Git 사용법

- 프로젝트 파일을 생성할 경로에 폴더 하나 생성
ex) `c:/dmu/project/`
- 경로에서 git bash를 사용해 `git init` 명령어 실행하여 git 연동

```
# 프로젝트 파일에서 cmd or power shell 열기
# 명령어는 한줄씩 입력하고 문제시 "git status"입력후 텍스트 카톡에 올리기
git init
git remote add origin https://github.com/DMU-NextLevel/demo-repository.git
```

```

git pull
git pull origin main

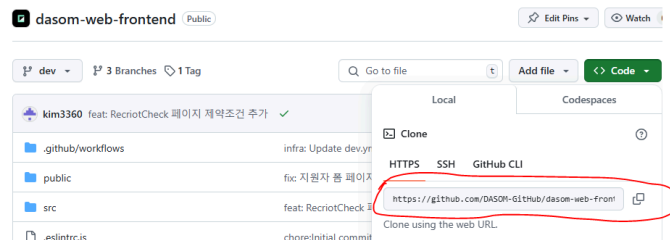
# git 을 처음 쓰는 컴퓨터만 실행 - git global 설정
# git 정보와 다른 정보 입력시 본인의 github에 로그가 남지 않음
git config --global user.name "git nick name 값"
git config --global user.email "git 이메일"

# git 작업 시작 전
git pull origin main

# git 작업 끝난 이후
git add .
git commit # git commit -m "commit 간단한 message"
# 이후 vi편집기로 commit message작성
# esc키, ":wq enter"
git push origin main

# git 충돌 발생시
git add .
git commit -m "git 충돌 발생"
git pull origin main
# git 오류 내용중 어떤 파일에서 충돌이 발생했는지 확인
# 충돌 발생 code 확인하고 git 주석 지우기
git commit -m "git 충돌 해결"
git push origin main

```



git Branch 이동

1. Issue를 시작하기 전, dev 브랜치로 이동

- `git checkout dev`

2. dev 브랜치의 최신 개발 상태 연동

- `git pull`

3. 이슈 번호에 맞는 개발 브랜치 새로 생성

- `git checkout -b feat/#12`

4. 개발중 변경사항 하나마다 commit 메세지 작성 후 push

- `git add .` → git 로컬 브랜치에 현재 변경사항 전체 추가
- `git commit -m {commit메세지}` → 현재 저장한 변경사항 commit, commit메세지 추가
- `git push` → github 브랜치(원격 브랜치)에 현재 로컬브랜치에 commit된 내용들 추가



vscode에서도 같은 기능 사용가능, 자세한 내용은 구글링하세요
